PYTHON IF…ELSE

Programming for Data Science

Agenda

- Conditional Expressions & Comparisons
- if Statement
- if … else and elif
- Short-Hand if … else
- Logical Operators
- Nested if Statements
- pass Statement
- match Statement

If ... Else

- Python supports the usual logical conditions from mathematics:
  - Equals: a == b
  - Not Equals: a != b
  - Less than: a < b
  - Less than or equal: a <= b
  - Greater than: a > b
  - Greater than or equal: a >= b

If ... Else

```
a = 33
b = 200
if b > a:
  print("b is greater than a")
```

If ... Else

```
a = 33
b = 200
if b > a:
  print("b is greater than a")
```

Output

```
b is greater than a
```

If ... Else

```
a = 33
b = 200
if b > a:
  print("b is greater than a")
```

Output

```
b is greater than a
```

```
a = 33
b = 200
if b > a:
print("b is greater than a")
```

Else

```
a = 200
b = 33
if b > a:
  print("b is greater than a")
else:
  print("a is greater than b")
```

Output

```
a is greater than b
```

Else

```
a = 33
b = 33
if b > a:
  print("b is greater than a")
elif a == b:
  print("a and b are equal")
```

Output

```
a and b are equal
```

```
a = 200
b = 33
if b > a:
  print("b is greater than a")
elif a == b:
  print("a and b are equal")
else:
  print("a is greater than b")
```

Output

```
a is greater than b
```

Short Hand If, If … Else

```
a = 200
b = 33
if a > b: print("a is greater than b")
```



Output

```
a is greater than b
```

Short Hand If, If … Else

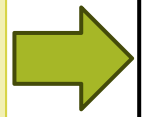
```
a = 200
b = 33
if a > b: print("a is greater than b")
```



Output

```
a is greater than b
```

```
a = 2
b = 330
print("A") if a > b else print("B")
```



Output

```
B
```

Short Hand If, If … Else

```
a = 200
b = 33
if a > b: print("a is greater than b")
```

Output
a is greater than b

```
a = 2
b = 330
print("A") if a > b else print("B")
```

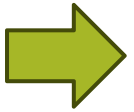
Output
B

```
a = 330
b = 330
print("A") if a > b else print("=") if a == b else print("B")
```

Output
=

Logical Operators

```
a = 200
b = 33
c = 500
if a > b and c > a:
  print("Both conditions are True")
```



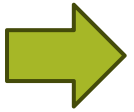
Output
Both conditions are True

```
a = 200
b = 33
c = 500
if a > b or a > c:
  print("At least one of the conditions is True")
```



Output
At least one of the conditions is True

```
a = 33
b = 200
if not a > b:
  print("a is NOT greater than b")
```



Output
a is NOT greater than b

Nested If

```
x = 41

if x > 10:
  print("Above ten,")
  if x > 20:
    print("and also above 20!")
  else:
    print("but not above 20.")
```

Output

```
Above ten,
and also above 20!
```

Pass Statement

```
a = 33
b = 200

if b > a:
  pass
else:
  print("printing else")

print("without printing else")
```

Output

```
without printing else
```

Match Statement

- The match statement is used to perform different actions based on different conditions.
- The match statement selects one of many code blocks to be executed.

```
match expression:
  case x:
    code block
  case y:
    code block
  case z:
    code block
```

Match Statement

```
day = 4
match day:
  case 6:
    print("Today is Saturday")
  case 7:
    print("Today is Sunday")
  case _:
    print(“This is Weekend")
```

Output

```
This is Weekend
```

Match Statement

```
month = 5
day = 4
match day:
  case 1 | 2 | 3 | 4 | 5 if month == 4:
    print("A weekday in April")
  case 1 | 2 | 3 | 4 | 5 if month == 5:
    print("A weekday in May")
  case _:
    print("No match")
```

Output

```
A weekday in May
```

Agenda

- Introduction to Loops
- for Loops
- enumerate() with for Loops
- break and continue
- range() Function
- for … else
- Nested for Loops
- pass in for Loops
- while Loops
- while with break and continue
- while … else

Loops

- Python has two primitive loop commands:
  - ***while*** loops
  - ***for*** loops

“for” Loops

- A “for” loop is used for iterating over a sequence (that is either a list, a tuple, a dictionary, a set, or a string).
- With the “for” loop we can execute a set of statements, once for each item in a list, tuple, set etc.

“for” Loops

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
  print(x)
```

```
Output
apple
banana
cherry
```

```
for x in "banana":
  print(x)
```

```
Output
b
a
n
a
n
a
```

“for” Loops (Enumerate)

```
fruits = ["apple", "banana", "cherry"]
for i, val in enumerate(fruits):
  print(i, val)
```



```
Output
0 apple
1 banana
2 cherry
```

“for” Loops (break & continue)

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
  print(x)
  if x == "banana":
    break
```



Output
apple
banana

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
  if x == "banana":
    break
  print(x)
```



Output
apple

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
  if x == "banana":
    continue
  print(x)
```



Output
apple
cherry

“for” Loops (range() func.)

```
for x in range(6):
  print(x)
```



```
Output
0
1
2
3
4
5
```

```
for x in range(2, 6):
  print(x)
```



```
Output
2
3
4
5
```

```
for x in range(2, 10, 3):
  print(x)
```



```
Output
2
5
8
```

“for” Loops (Else)

The else keyword in a for loop specifies a block of code to be executed when the loop is finished.

```
for x in range(6):
  print(x)
else:
  print("Finally finished!")
```



```
Output
0
1
2
3
4
5
Finally finished!
```

```
for x in range(6):
  if x == 3: break
  print(x)
else:
  print("Finally finished!")
```



```
Output
0
1
2
```

The “else” block will NOT be executed if the loop is stopped by a “break” statement.

“for” Loops (Nested)

A nested loop is a loop inside a loop.

The "inner loop" will be executed one time for each iteration of the "outer loop"

```
adj = ["big", "tasty"]
fruits = ["apple", "banana", "cherry"]

for x in adj:
  for y in fruits:
    print(x, y)
```

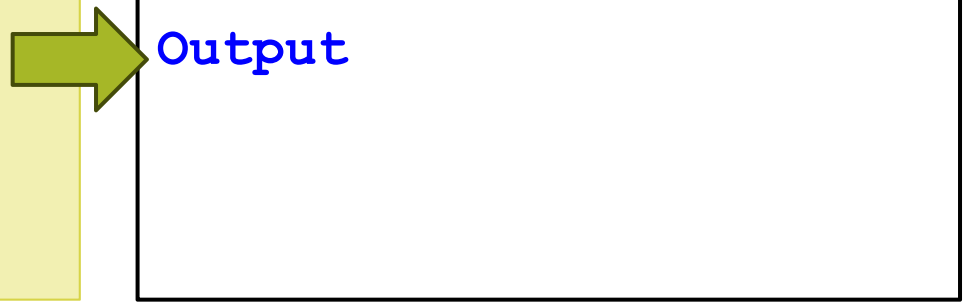


```
Output
big apple
big banana
big cherry
tasty apple
tasty banana
tasty cherry
```

“for” Loops (“pass”)

“for” loops cannot be empty, but if you for some reason have a “for” loop with no content, put in the “pass” statement to avoid getting an error.

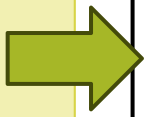
```
for x in [0, 1, 2]:
  pass
```



“while” Loops

With the “while” loop we can execute a set of statements as long as a condition is true.

```
i = 1
while i < 6:
  print(i)
  i += 1
```



```
Output
1
2
3
4
5
```

Remember to increment “I”, or else the loop will continue forever.

“while” Loops (break & continue)

```
i = 1
while i < 6:
  print(i)
  if i == 3:
    break
  i += 1
```



```
Output
1
2
3
```

```
i = 0
while i < 6:
  i += 1
  if i == 3:
    continue
  print(i)
```



```
Output
1
2
4
5
6
```

“while” Loops (Else)

```
i = 1
while i < 6:
  print(i)
  i += 1
else:
  print("i is no longer less than 6")
```



```
Output
1
2
3
4
5
i is no longer less than 6
```

With the “else” statement we can run a block of code once when the condition no longer is true

Questions?

Thank you!